

BÁO CÁO BÀI TẬP LỚN

HỆ THỐNG ĐẤU GIÁ TRỰC TUYẾN

UET AUCTION SYSTEM

Môn học: Lập trình nâng cao (2526II_UET.CS2043_1)

Nhóm thực hiện: Nhóm 10

Thành viên : Phùng Anh Dũng (25020077)

Đậu Đình Gia Bảo (25020034)

Nguyễn Minh Nhật Anh (25020022)

Nguyễn Viết Hưng (25020196)

1. Giới thiệu mục tiêu và phạm vi thực hiện.

1.1. Mục tiêu dự án.

Xây dựng ứng dụng Desktop mô phỏng hệ thống đấu giá trực tuyến thời gian thực (Real-time) theo mô hình Client-Server, đảm bảo:

- **Real-time:** Cập nhật giá thầu và trạng thái phiên đấu giá tức thời đến mọi client không qua tải lại trang.
- **Concurrency:** Xử lý an toàn tranh chấp khi nhiều người dùng cùng đặt giá tại một mili-giây, ngăn chặn mất mát dữ liệu (Lost Update).
- **UX/UI chuyên nghiệp:** Giao diện trực quan (JavaFX), tích hợp ví điện tử nội bộ và trực quan hóa lịch sử đặt giá bằng biểu đồ.

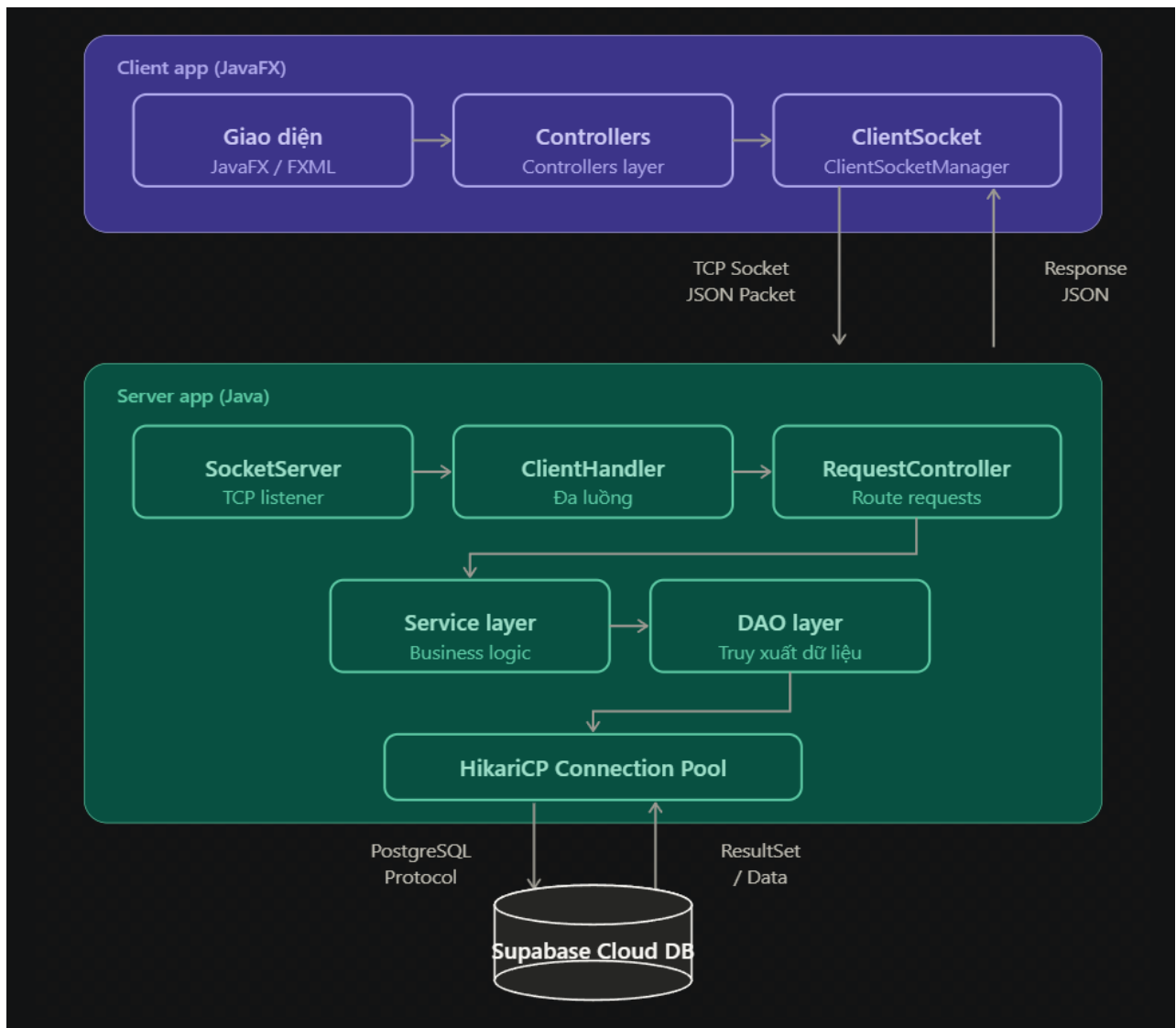
1.2. Phạm vi hệ thống.

Hệ thống bao gồm các phân hệ nghiệp vụ chính sau:

- **Quản trị & Phân quyền:** Phân vai rõ ràng giữa Admin (Quản trị hệ thống/phiên đấu giá) và Member (Người bán/Người mua).
- **Quản lý phiên đấu giá:** Khởi tạo, theo dõi và vận hành các phiên đấu giá theo danh mục sản phẩm (Nghệ thuật, Phương tiện, Điện tử).
- **Cơ chế đấu giá nâng cao:** Hỗ trợ đặt giá thủ công, tự động (Auto-Bidding) và chống đầu cơ giây cuối (Anti-sniping).
- **Giao dịch & Thanh toán:** Ví điện tử cá nhân (nạp, rút) và tự động khấu trừ ví chuyển tiền (Auto-billing) khi kết thúc phiên.
- **Môi trường & Hạ tầng:** Giao tiếp mạng TCP Socket truyền tải gói JSON. Cơ sở dữ liệu đám mây PostgreSQL (Supabase).

2. Thiết kế kiến trúc hệ thống.

- Hệ thống được thiết kế theo mô hình kiến trúc **Client-Server phân lớp (Layered Architecture)** giao tiếp qua **TCP Socket** truyền tải dữ liệu định dạng **JSON**.



3. Mô tả luồng hoạt động của hệ thống.

Luồng hoạt động của hệ thống được vận hành khép kín thông qua hai chu trình chính:

3.1. Luồng gửi yêu cầu.

- **Tương tác:** Người dùng thực hiện thao tác trên giao diện JavaFX/FXML.
- **Xác thực & Đóng gói:** Controllers xác thực dữ liệu đầu vào và đóng gói thành đối tượng DTO.
- **Tuần tự hóa:** ClientSocketManager chuyển đổi DTO thành chuỗi JSON thông qua thư viện Gson.
- **Truyền tải:** Gói tin JSON được gửi đến Server qua kết nối TCP Socket.
- **Định tuyến:** SocketServer tiếp nhận kết nối và phân phối cho luồng ClientHandler tương ứng; RequestController giải mã và định tuyến yêu cầu đến đúng Service.
- **Thực thi nghiệp vụ:** Service xử lý logic nghiệp vụ, gọi DAO để thực thi truy vấn xuống Supabase Cloud DB thông qua HikariCP Connection Pool.

3.2. Luồng phản hồi và cập nhật.

- **Phản hồi trực tiếp (Unicast):** Server đóng gói kết quả truy vấn và gửi phản hồi về đúng Client thông qua kết nối Socket riêng; Controller nhận và cập nhật giao diện tương ứng (ví dụ: đăng nhập thành công, nạp tiền thành công).

- **Phát tin thời gian thực (Broadcast):** Khi xảy ra các sự kiện thay đổi trạng thái chung (ví dụ: có lượt đặt giá mới, gia hạn phiên đấu giá), Server tự động phát gói tin cập nhật tới tất cả Client liên quan.

- **Cập nhật giao diện:** Client nhận gói tin JSON, giải tuần tự hóa và cập nhật giao diện ngay lập tức (vẽ lại biểu đồ giá, hiển thị thông báo).

4. Chi tiết các chức năng đạt được, hướng giải quyết và lý do lựa chọn.

Tên chức năng & Tiêu chí barem	Lớp / Thành phần phụ trách chính	Hướng giải quyết	Lý do lựa chọn
1. Quản lý người dùng (BẮT BUỘC) - Đăng ký & đăng nhập tài khoản - Phân quyền hai vai trò tối giản - Tích hợp tính năng mua/bán	UserService, UserDAOImpl, UserRole, AccountHandler, LoginController, SessionManager	- Xác thực và lưu mật khẩu thông qua UserService. - Áp dụng mô hình phân quyền tối giản với 2 Role: ADMIN và MEMBER. - Gộp chung nghiệp vụ Người bán và Người đấu giá vào cùng tài khoản MEMBER.	- Trải nghiệm người dùng (UX): Tối ưu luồng người dùng, tích hợp Mua/Bán trên một tài khoản. - Khía cạnh kỹ thuật: Giảm độ phức tạp CSDL và tối giản hóa quản lý Session.
2. Quản lý sản phẩm (BẮT BUỘC) - CRUD sản phẩm đấu giá - Phân loại danh mục OOP - Thiết lập giá & thời hạn phiên	Item (Abstract), Art/Electronics/ Vehicle, ItemFactory, ItemService, ItemDAOImpl, ManageSellerController	- Thiết kế kế thừa (Inheritance): Lớp trừu tượng Item -> các lớp con Art, Electronics, Vehicle. - Khởi tạo đối tượng động thông qua Factory Method (ItemFactory) dựa trên ItemType đầu vào.	- Hướng đối tượng (OOP): Tuân thủ nguyên lý Đa hình và Đóng-Mở (OCP). - Khả năng mở rộng: Dễ dàng bổ sung danh mục mới không ảnh hưởng mã nguồn lõi.
3. Đấu giá & Vòng đời phiên (BẮT BUỘC) - Đặt giá thầu hợp lệ - Cập nhật người dẫn đầu phiên - Quét ngầm đóng/mở phiên	AuctionService, BidService, AuctionScheduler, PaymentService, AuctionDAOImpl	- BidService kiểm duyệt logic bước giá và ghi nhận người dẫn đầu (Winner) ngay tức thời. - Sử dụng ScheduledExecutorService (Luồng ngầm AuctionScheduler) chạy định kỳ 30s/lần để quét và	- Tính tự động hóa: Khép kín vòng đời phiên và thanh toán mà không cần can thiệp thủ công. - Quản lý trạng thái chủ động, độc lập thông qua Timer chạy ngầm.

-Thanh toán tự động khi kết thúc		chuyển trạng thái phiên (OPEN -> RUNNING -> FINISHED).	
4. Xử lý lỗi & Ngoại lệ (BẮT BUỘC) - Chặn đặt giá thấp hơn giá hiện tại - Chặn đặt giá khi phiên đã đóng - Xử lý lỗi số dư ví, lỗi kết nối	ValidationException, ValidationUtils, AlertUtils, ClientSocketManager	- Chuẩn hóa kiểm tra ràng buộc bằng ValidationUtils, nếu vi phạm sẽ ném ngoại lệ ValidationException . - Server trả về mã lỗi thông qua giao thức Socket, Client dùng AlertUtils để hiện Pop-up cảnh báo.	- Toàn vẹn dữ liệu: Ngăn chặn triệt để giao dịch rác từ Server. - Độ ổn định hệ thống (Stability): Bắt lỗi ngoại lệ linh hoạt, tránh tình trạng văng ứng dụng (Crash).
5. Giao diện người dùng (BẮT BUỘC) - Màn hình Danh sách phiên đấu - Màn hình Chi tiết đấu giá trực tiếp - Màn hình Ví điện tử & nạp tiền - Màn hình Quản lý kho của Seller	JavaFX / FXML, MainController, AuctionListController, AuctionDetailController, ProductCardController	- Lập trình UI bằng JavaFX & FXML, phân tách mã nguồn thành mô hình MVC. - Tái sử dụng UI Component (như ProductCardController cho từng mặt hàng).	- Trải nghiệm: Đảm bảo phản hồi nhanh, UI đồng bộ, trực quan trên Desktop. - Dễ dàng bảo trì: Phân tách rõ ràng giao diện/ng nghiệp vụ (MVC), tối ưu bảo trì.
6. Đấu giá đồng thời (NÂNG CAO) - Loại bỏ lỗi Lost Update (ghi đè) - Tránh Rollback lỗi do tranh chấp - Đảm bảo duy nhất 1 người thắng	LockManager, Khối đồng bộ synchronized, DatabaseConnection (HikariCP)	- Fine-grained Locking: Lớp LockManager cấp phát riêng một Object Lock cho từng auctionId để dùng trong khối synchronized của luồng đặt giá. - Lưu trữ tuần tự xuống CSDL thông qua Connection Pool HikariCP.	- An toàn giao dịch : Triệt tiêu lỗi tranh chấp dữ liệu (Race Condition) khi lượng truy cập cực lớn. - Hiệu năng: Tránh gây nghẽn cổ chai Server.
7. Cập nhật thời gian thực (NÂNG CAO) - Đẩy dữ liệu thầu mới lập tức - Đồng bộ thời gian đếm ngược	AuctionManager, ClientHandler, AuctionEventDispatcher	- Ứng dụng Observer Pattern trên nền mạng TCP Socket. Khi có Bid mới, Server gọi notifyObservers() đẩy gói tin JSON Broadcast đến các Client đang mở phiên.	- Tối ưu đường truyền: Giải quyết triệt để tình trạng lãng phí tài nguyên. - Thời gian thực (Real-time): Đảm bảo độ trễ

- Không sử dụng HTTP Polling		- Client có luồng SocketReader ngầm nhận tin và vẽ lại UI.	cập nhật trạng thái đấu giá dưới 1 giây.
8. Đấu giá tự động (NÂNG CAO) - Thiết lập giá tối đa & bước giá - Tự động nâng giá thay bidder - Đua giá tự động giữa các cấu hình - Ưu tiên theo thời gian đăng ký	AutoBidService, AutoBidDAOImpl, AutoBidEntry, PriorityQueue	- Lưu cấu hình thầu (maxBid, increment) qua AutoBidService. - Sử dụng PriorityQueue (Hàng đợi ưu tiên) sắp xếp thời gian cấu hình (FIFO). Thuật toán tự so khớp và tự động phát lệnh Bid thay mặt người dùng cho đến khi chạm trần maxBid.	- Tối đa hóa trải nghiệm, hỗ trợ khách hàng không thể trực tuyến liên tục. - Bảo mật định mức thầu, duy trì sự tăng trưởng giá trị tài sản liên tục.
9. Chống bắn tỉa giây cuối (NÂNG CAO) - Phát hiện đặt thầu sát giờ hết hạn - Tự động kéo dài thời gian phiên - Chặn bot săn thầu giây cuối	BidService, AuctionUtils, AuctionDAOImpl	- Hàm AuctionUtils.applyAntiSnipe được gọi sau mỗi lệnh bid thành công. Nếu lệnh bid xảy ra trong vòng 30 giây cuối cùng, thuật toán tự động cộng thêm 60 giây vào thời hạn phiên và Broadcast cho mọi Client.	- Chống gian lận: Ngăn chặn các công cụ Bot (Sniping) cướp thầu vào giây cuối. - Công bằng: Tạo môi trường cạnh tranh minh bạch, gia tăng lợi nhuận cho người bán.
10. Biểu đồ lịch sử giá (NÂNG CAO) - Vẽ biểu đồ đường biến động giá • Trực quan hóa giá theo thời gian - Tự động cập nhật điểm đồ thị	LineChart (JavaFX), AuctionDetailController, Platform.runLater()	- Tích hợp Component LineChart của JavaFX vào giao diện. Trục X là Thời gian, Trục Y là Giá tiền. - Mỗi khi Socket nhận được điểm dữ liệu giá mới, sử dụng Platform.runLater() để nhúng vào luồng Main UI vẽ thêm nút đồ thị.	- Giao diện & Trải nghiệm (UI/UX): Trực quan hóa đường cong biến động giá một cách chuyên nghiệp. - Phân tích: Hỗ trợ người dùng đánh giá nhanh mức độ sôi động của phiên thầu.
11. Ví điện tử & Tự thanh toán (NÂNG CAO) - Nạp/Rút tiền ảo vào ví cá nhân - Phong tỏa tạm giữ (Hold Balance) - Tự động trừ ví/chuyển ví khi đóng	WalletService, PaymentService, BidTransactionDAOImpl	- WalletService tính toán và phong tỏa (Hold Balance) số dư khả dụng đối với những phiên mà user đang là người thắng cao nhất. - Nếu số dư trừ đi phần đang bị giữ không đủ, lệnh Rút tiền (Withdraw) bị	- Quản trị rủi ro: Triệt tiêu hoàn toàn rủi ro người thắng 'bùng tiền' không thanh toán. - Tin cậy: Tiền được phong tỏa từ đầu, bảo vệ tuyệt đối quyền lợi của Seller.

- Không thể rút tiền khi đang thắng		chặn. Khi phiên đóng, tiền hành trừ tiền tự động.	
-------------------------------------	--	---	--

5. Bảng phân công nhiệm vụ chi tiết

Họ và tên	Mã sinh viên	Github	Công việc chính	Chi tiết công việc
Phùng Anh Dũng	25020077	padungg	Server Logic & Socket Server	Xây dựng Socket Server đa luồng, điều phối logic và xử lý nghiệp vụ trung tâm phía Server. Tham gia thiết kế gói Model chung.
Nguyễn Minh Nhật Anh	25020022	nhatdog34	Database, Testing & CI/CD	Thiết kế CSDL (PostgreSQL/Supabase, DAO Layer), viết Unit Test, thiết lập GitHub Actions CI/CD. Hỗ trợ Dũng xử lý Server khi cần. Tham gia thiết kế gói Model chung.
Nguyễn Việt Hưng	25020196	ngvh2312	Client Logic & Socket Client	Xây dựng Socket Client kết nối tới Server, xử lý truyền nhận dữ liệu JSON/Gson và lập trình Controller điều khiển sự kiện UI. Tham gia thiết kế gói Model chung.
Đậu Đình Gia Bảo	25020034	baothebean	Giao diện (UI/UX FXML)	Thiết kế toàn bộ giao diện JavaFX bằng các file .fxml, tổ chức tài nguyên hình ảnh, đảm bảo UI/UX. Hỗ trợ Hưng xử lý code phía Client. Tham gia thiết kế gói Model chung.